

Sprawozdanie

Password Break

14 czerwca 2026

Spis treści

1	Wstęp	2
1.1	Charakterystyka problemu	2
1.2	Cel projektu	2
1.3	Podstawy teoretyczne	2
2	Założenia projektowe	2
2.1	Zakres funkcjonalny	2
2.2	Przypadki użycia	3
2.3	Wybór technologii	3
2.4	Komponenty systemu	3
2.5	Konfiguracja	4
3	Architektura systemu	4
3.1	Najważniejsze klasy	4
4	Komunikacja	5
4.1	Kontrakt — Protocol Buffers	5
4.2	Przebieg komunikacji	6
4.3	Heartbeat, timeout i ponowny przydział	6
4.4	Synchronizacja słownika	7
5	Jak ogólnie działa projekt	7
5.1	Cykl życia serwera i sterowanie przebiegiem	7
5.2	Podział pracy na zadania	7
5.3	Numerowanie kandydatów w trybie brute force	7
5.4	Obliczenia po stronie klienta (wielowątkowość)	8
5.5	Pomiar metryk	8
6	Metodologia eksperymentów i wyniki	9
6.1	Metodologia	9
6.2	Wpływ liczby klientów	9
6.3	Wpływ granulacji i liczby klientów	10
6.4	Wcześniej zauważone problemy i ich usunięcie	10
7	Wnioski końcowe	10

1 Wstęp

1.1 Charakterystyka problemu

Hasła w rzeczywistych systemach nie są przechowywane w postaci jawnej, lecz jako wynik działania kryptograficznej funkcji skrótu. System uwierzytelniający nie zna bezpośrednio tekstu hasła, a jedynie jego skrót. Projektowany program nie odwraca funkcji haszującej — co jest obliczeniowo niewykonalne — lecz *generuje* kolejnych kandydatów na hasło, oblicza ich skróty i porównuje je ze skrótem docelowym.

Problem można rozproszyć na wiele komputerów: poszczególni klienci niezależnie sprawdzają różne fragmenty przestrzeni możliwych haseł, a serwer przydziela pracę i zbiera wyniki. Sprawdzanie kolejnych haseł nie zależy od siebie nawzajem, więc obliczenia różnych klientów nie wymagają wzajemnej synchronizacji.

1.2 Cel projektu

Celem projektu jest zbudowanie działającego systemu rozproszonego i analiza jego wydajności przy różnych konfiguracjach pracy. System obejmuje:

- komunikację klient–serwer w trybie dwukierunkowego strumienia,
- dynamiczny przydział i synchronizację pracy,
- obsługę błędów (rozłączenia, zawieszenia klientów, ponowny przydział zadań),
- pomiar i analizę wydajności (przepustowość, skalowalność, wpływ granulacji).

Mierzone są między innymi: liczba sprawdzonych kandydatów na sekundę, wpływ liczby klientów na wydajność, wpływ granulacji (wielkości paczek zadań) oraz narzut komunikacji względem czasu właściwych obliczeń.

1.3 Podstawy teoretyczne

Funkcja skrótu SHA-256. SHA-256 przekształca dane wejściowe dowolnej długości w ciąg o stałej długości 256 bitów (32 bajty). Kluczowe właściwości wykorzystywane w projekcie to:

- *deterministyczność* — te same dane zawsze dają ten sam skrót,
- *jednokierunkowość* — nie da się praktycznie odtworzyć danych ze skrótu,
- *odporność na kolizje* oraz *efekt lawinowy* — minimalna zmiana wejścia całkowicie zmienia skrót.

Najdroższą operacją jest samo haszowanie, dlatego opłaca się porównywać jeden skrót z wieloma kandydatami, a target hashe trzymać w strukturze `HashSet` o stałym czasie wyszukiwania.

Atak brute force. Metoda generuje wszystkie możliwe kombinacje znaków z ustalonego alfabetu (małe litery, cyfry, ewentualnie znaki specjalne) w zadanym zakresie długości. Jest pewna, ale kosztowna obliczeniowo. Aby podzielić pracę, wszystkie możliwe hasła zostają *ponumerowane*, a serwer przydziela klientom zakresy indeksów do sprawdzenia (patrz rozdz. 5.3).

Atak słownikowy. Metoda sprawdza hasła z przygotowanego pliku (słownika) zawierającego często spotykane słowa i ich warianty (dodane cyfry, zmiana wielkości liter, znaki specjalne, podstawienia typu $a \rightarrow @$, $o \rightarrow 0$). Pozwala szybko znaleźć słabe hasła. W systemie rozproszonym słownik dzielony jest na zakresy indeksów tak samo jak w trybie brute force.

2 Założenia projektowe

2.1 Zakres funkcjonalny

- Aplikacja serwera uruchamiana z konsoli, z parametrami zadania w pliku konfiguracyjnym.

- Aplikacja klienta uruchamiana z konsoli, z adresem serwera podawanym jako argument.
- Obsługa dwóch trybów pracy: *brute force* oraz *słownikowego*.
- Dynamiczny przydział paczek zadań klientom, którzy zgłoszą gotowość.
- Wielowątkowe obliczenia po stronie klienta.
- Logowanie wyników oraz metryk wydajności do plików CSV po stronie serwera.
- Możliwość uruchamiania na różnej liczbie klientów (skalowanie poziome).

2.2 Przypadki użycia

1. Użytkownik uruchamia serwer i ustawia parametry zadania (tryb, alfabet, długości, skróty docelowe).
2. Użytkownik uruchamia klientów na kolejnych komputerach, wskazując adres serwera.
3. Klient łączy się, pobiera konfigurację i oczekuje na sygnał startu.
4. Po starcie klient pobiera paczkę pracy, wykonuje obliczenia i odsyła wynik, po czym pobiera następną.
5. Klient znajduje poprawne hasło i raportuje je do serwera.
6. Serwer zapisuje końcowy raport i metryki testu.

2.3 Wybór technologii

Obszar	Technologia
Platforma / język	.NET, C#
Komunikacja sieciowa	gRPC (HTTP/2), Protocol Buffers (proto3)
Serwer	ASP.NET Core gRPC Service
Klient, generator, Plots	aplikacje konsolowe .NET
Wykresy	ScottPlot
Środowisko	Nix flake (dev-shell)

W projekcie zastosowano architekturę klient-serwer: jeden węzeł pełni rolę koordynatora (serwer), pozostałe wykonują obliczenia (klienci). gRPC wybrano jako nowsze rozwiązanie oraz dlatego, że podejście *contract-first* (wspólny plik `.proto`, z którego generowany jest kod serwera i klienta) jest prostsze niż budowanie API w REST. Dodatkowo gRPC ma wbudowaną obsługę strumieni dwukierunkowych, których ten system potrzebuje.

2.4 Komponenty systemu

Repozytorium zawiera rozwiązanie `password-break.slnx` złożone z następujących projektów:

`password-break-server`

Serwer koordynujący: dzieli przestrzeń przeszukiwania na zadania, przydziela je klientom, zbiera wyniki, mierzy i zapisuje metryki.

`password-break-client`

Klient roboczy: łączy się z serwerem, pobiera zadania, oblicza skróty wielowątkowo i odsyła wyniki.

`password-break-monitor`

Dodatkowe narzędzie do podglądu stanu systemu.

`password-break-wordlist-generator`

Generator słownika na potrzeby trybu słownikowego.

`Plots`

Narzędzie analityczne: czyta pliki `metrics.csv` i generuje wykresy wydajności (ScottPlot).

2.5 Konfiguracja

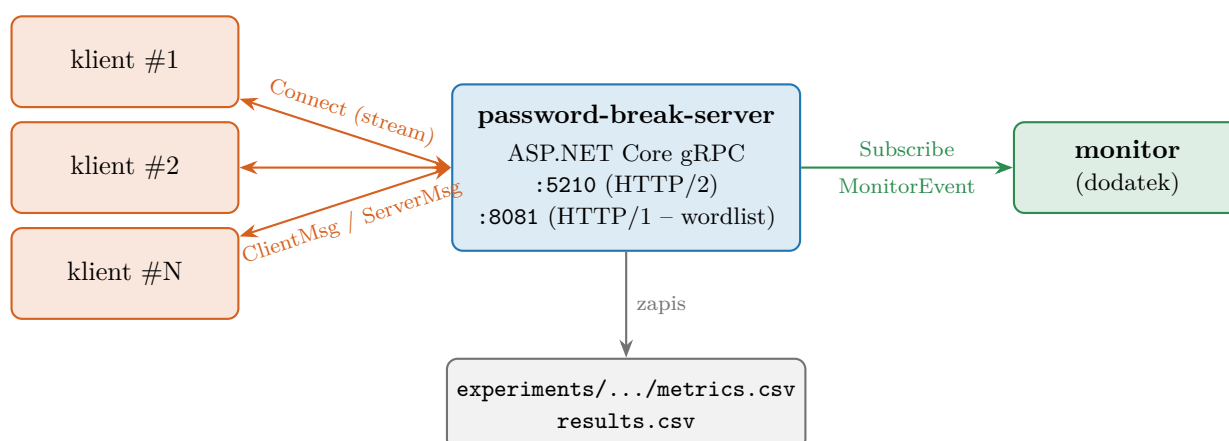
Parametry pracy serwera znajdują się w pliku `appsettings.json` (sekcja `PasswordBreak`). Najważniejsze pola odpowiadają klasie `PasswordBreakConfig`:

```
"PasswordBreak": {
  "AttackMode": "bruteforce",           // "bruteforce" lub "dictionary"
  "CharSet": "abcdefghijklmnopqrstuvwxyz0123456789",
  "MinLength": 1, "MaxLength": 8,
  "ChunkSize": 100000,                 // liczba kandydatów na jedno zadanie (granulacja)
  "WordListPath": "wordlist.txt",
  "HeartbeatIntervalSeconds": 5,       // co ile klient wysyła heartbeat
  "HeartbeatTimeoutSeconds": 20,       // brak heartbeatu => klient uznany za rozłączonego
  "TaskTimeoutSeconds": 60,            // zadanie zbyt długo InProgress => requeue
  "TargetHashes": [ "5e884898...", "8c6976e5...", "6b86b273..." ]
}
```

Listing 1: Fragment konfiguracji serwera (`appsettings.json`).

3 Architektura systemu

System składa się z jednego serwera oraz dowolnej liczby klientów. Dodatkowo do serwera może podłączyć się monitor, który tylko *obserwuje* stan (nie wykonuje obliczeń). Rysunek 1 przedstawia ogólną architekturę i kanały komunikacji.



Rysunek 1: Architektura systemu: serwer koordynuje N klientów (dwukierunkowy strumień gRPC) oraz strumieniuje zdarzenia do monitora. Wyniki i metryki zapisywane są po stronie serwera.

3.1 Najważniejsze klasy

Serwer (`password-break-server`).

`PasswordBreakerService`

Usługa gRPC obsługująca dwukierunkowy strumień `Connect`. Odbiera wiadomości klienta, przydziela zadania, przyjmuje wyniki, prowadzi watchdog heartbeatu i zapisuje metryki.

`TaskManager (ITaskManager)`

Zarządza pulą pracy: dzieli przestrzeń przeszukiwania na chunki, wydaje kolejne zadania, śledzi zadania aktywne i oczekujące oraz zwraca je do kolejki (`requeue`) po awarii lub timeoucie.

`FoundPasswords`

Przechowuje znalezione hasła, śledzi skróty jeszcze nieznalezione, sygnalizuje znalezienie wszystkich (`AllFound`) i zapisuje wyniki.

ClientTracker

Rejestr połączonych klientów (adres, czas ostatniego heartbeatu), używany do wykrywania rozłączeń.

ExperimentRunManager

Obsługuje pojedyncze przebiegi eksperymentu — katalog wyników, pliki `metrics.csv/results.csv` oraz numer przebiegu.

ServerRunState

Stan serwera RUNNING/PAUSE przełączany SPACJĄ (Toggle).

TaskTimeoutChecker

Usługa działająca w tle; cyklicznie zwraca do kolejki zadania zbyt długo będące w stanie *InProgress*.

PasswordBreakConfig, TaskInfo

Modele: konfiguracja ataku (tryb, alfabet, długości, `ChunkSize`, skróty, timeouty) oraz pojedyncze zadanie (identyfikator, zakres indeksów, status, przypisany klient).

Klient (password-break-client).

GrpcClient

Łączy się z serwerem i utrzymuje połączenie: pętla odbioru wiadomości, pętla heartbeatu, przetwarzanie `HashTask` i odsyłanie `Result`, automatyczne ponawianie połączenia po rozłączeniu.

IClientAttackStrategy (BruteForceClientStrategy, DictionaryClientStrategy)

Strategia ataku wybierana na podstawie konfiguracji z serwera.

HashWorker

Klasa statyczna z rdzeniem obliczeń: liczenie SHA-256, równoległe przetwarzanie zakresu (brute force/słownik) oraz odtwarzanie hasła z indeksu.

WordlistManager (IWordlistManager)

Pobiera (HTTP) i łączy słownik, zapewniając jego synchronizację między węzłami.

4 Komunikacja

4.1 Kontrakt — Protocol Buffers

Cała komunikacja jest opisana w pliku `Protos/password.proto`. Definiuje on dwie usługi: `PasswordBreaker` (praca rozproszona, strumień dwukierunkowy) oraz `Monitor` (strumień jednokierunkowy zdarzeń do monitora).

```
service PasswordBreaker {
  rpc Connect(stream ClientMessage) returns (stream ServerMessage);
}
message ClientMessage { oneof message { Heartbeat heartbeat=1; Result result=2;
                                     Ready ready=3; Hello hello=4; } }
message ServerMessage { oneof message { HashTask hash_task=1; Ack ack=2; Config config=3; } }

service Monitor { rpc Subscribe(SubscribeRequest) returns (stream MonitorEvent); }
```

Listing 2: Definicja usług i wiadomości (`password.proto`, `proto3`).

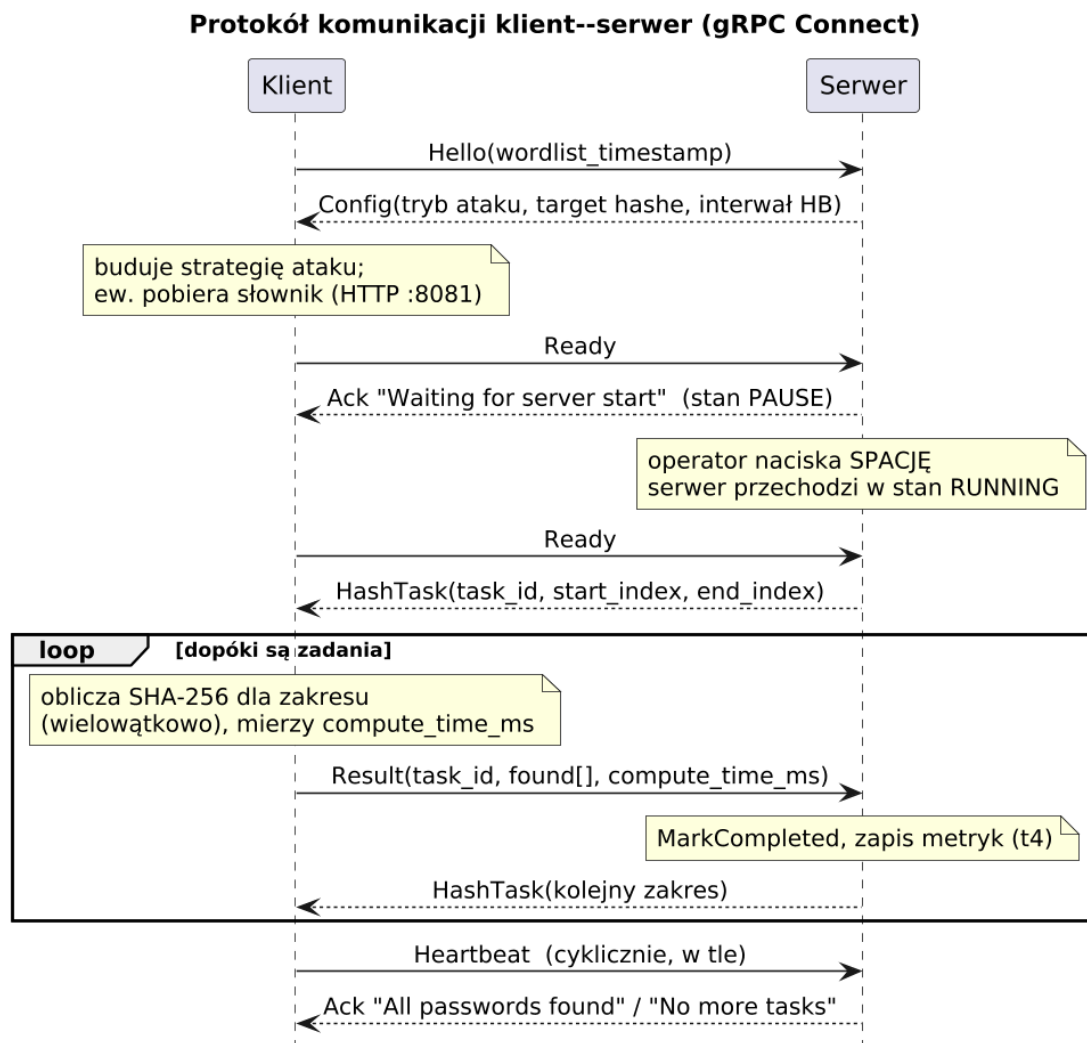
Wiadomości wykorzystują konstrukcję `oneof`, co odpowiada koncepcji komunikatów z wcześniejszych założeń projektu (TASK / RESULT / STOP), tu w wersji rozszerzonej:

- **Od klienta** (`ClientMessage`): `Hello` (zgłoszenie z znacznikiem czasu lokalnego słownika), `Ready` (gotowość do pracy), `Result` (wynik zadania wraz z czasem obliczeń), `Heartbeat` (sygnał „żyję”).

- **Od serwera (ServerMessage):** Config (parametry ataku, target hashe, interwał heartbeat), HashTask (zakres indeksów do sprawdzenia), Ack (potwierdzenie / sygnał stanu, np. „Waiting for server start”, „No more tasks”, „All passwords found”).

4.2 Przebieg komunikacji

Rysunek 2 pokazuje pełny cykl życia połączenia jednego klienta — od nawiązania strumienia, przez negocjację konfiguracji, oczekiwanie na start, aż po pętlę „zadanie → wynik”.



Rysunek 2: Diagram sekwencji komunikacji klient–serwer w ramach jednego strumienia Connect. Heartbeaty wysyłane są równoległe do głównej pętli pracy.

4.3 Heartbeat, timeout i ponowny przydział

Niezawodność systemu rozproszonego zapewniają trzy współpracujące mechanizmy:

- **Heartbeat klienta** — `GrpcClient.HeartbeatLoop` cyklicznie (interwał z Config) wysyła pustą wiadomość Heartbeat; serwer aktualizuje „ostatnio widziany” w `ClientTracker`.
- **Watchdog serwera** — dla każdego połączenia `PasswordBreakerService` uruchamia zadanie kontrolne; jeśli od klienta nie ma heartbeatu dłużej niż `HeartbeatTimeoutSeconds`, strumień zostaje zamknięty, a w `Cleanup` bieżące zadanie klienta wraca do kolejki (`MarkPending`).
- **Timeout zadania** — hostowana usługa `TaskTimeoutChecker` okresowo wywołuje `RequeueExpiredTasks`; zadanie pozostające w stanie `InProgress` dłużej niż `TaskTimeoutSeconds` jest zwracane do

kolejki, co chroni przed zacięciem klienta na pojedynczej paczce.

Dodatkowo wynik dla zadania spoza aktywnego „przebiegu” (lub już zakończzonego) jest ignorowany — identyfikator zadania i numer przebiegu (`RunSequence`) pełnią funkcję ochrony przed duplikatami.

4.4 Synchronizacja słownika

W trybie słownikowym klient w wiadomości `Hello` przesyła znacznik czasu posiadanej kopii słownika. Serwer w `Config` podaje znacznik aktualnej wersji. Jeśli znaczniki się różnią, klient pobiera słownik zwykłym żądaniem HTTP GET z endpointu `/wordlist` (port 8081), po czym ładuje go do pamięci. Dzięki temu wszystkie węzły operują na identycznym, uporządkowanym zbiorze słów, co jest warunkiem poprawnego podziału zakresowego.

Dodatkowo serwer udostępnia osobną usługę `Monitor` (`Subscribe`), która strumieniuje zdarzenia stanu systemu do opcjonalnego narzędzia podglądu.

5 Jak ogólnie działa projekt

5.1 Cykl życia serwera i sterowanie przebiegiem

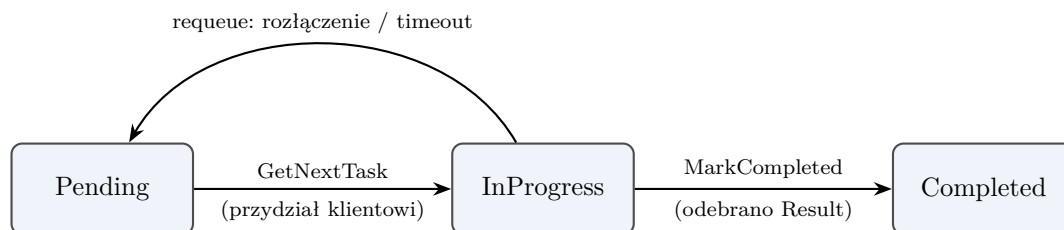
Po uruchomieniu serwer wchodzi w stan **PAUSE** — klienci mogą się łączyć i pobierać konfigurację, ale nie otrzymują zadań. Operator steruje przebiegiem z konsoli:

- **SPACJA** — przełącza stan `PAUSE` ↔ `RUNNING` (`ServerRunState.Toggle`). Przejście w `RUNNING` otwiera nowy „przebieg eksperymentu” (`ExperimentRunManager.StartNewRun`) i tworzy katalog wynikowy.
- **CTRL+C** — zatrzymuje serwer i zapisuje wyniki.

Każdy przebieg ma własny katalog `experiments/<nazwa>/` z plikami `metrics.csv` i `results.csv`; nazwa koduje liczbę klientów, liczbę wątków, `ChunkSize` oraz numer przebiegu, co ułatwia późniejszą analizę.

5.2 Podział pracy na zadania

`TaskManager` traktuje całą przestrzeń przeszukiwania jako uporządkowany zbiór ponumerowanych kandydatów (`_totalItems`). Przy każdym żądaniu `GetNextTask` wycina kolejny zakres o szerokości `ChunkSize` (lub zwraca zadanie odzyskane z kolejki `_pendingTasks`, jeśli jakieś wróciło po błędzie). Zadanie przechodzi przez stany przedstawione na rys. 3.



Rysunek 3: Stany pojedynczego zadania w `TaskManager`. Zadanie zwrócone do kolejki trafia ponownie do puli i może zostać przydzielone innemu klientowi.

5.3 Numerowanie kandydatów w trybie brute force

Dla każdej długości hasła kombinacje numerowane są jak liczby w systemie pozycyjnym o podstawie równej liczności alfabetu. Klient, otrzymawszy zakres indeksów `[start, end]`, wyznacza długość i indeks lokalny, a następnie odtwarza kolejne hasła:


```
private static string IndexToPassword(long index, string charset, int length)
{
    var result = new char[length];
    var charsetLength = charset.Length;

    for (var i = length - 1; i >= 0; i--)
    {
        result[i] = charset[(int)(index % charsetLength)];
        index /= charsetLength;
    }

    return new string(result);
}
```

Listing 3: Odtwarzanie hasła z indeksu — `HashWorker.IndexToPassword` (fragment pliku źródłowego).

5.4 Obliczenia po stronie klienta (wielowątkowość)

Po odebraniu `HashTask` klient dzieli przydzielony zakres na tyle podzakresów, ile wątków ma do dyspozycji (`MaxDegreeOfParallelism`) i przetwarza je równoległe przez `Parallel.ForEach`. Każdy wątek liczy SHA-256 kolejnych kandydatów i porównuje skrót ze zbiorem `targetHashes`. Znalezione hasła trafiają do `ConcurrentBag`. Co pewną liczbę iteracji sprawdzany jest token anulowania, dzięki czemu przerwanie połączenia szybko zatrzymuje pracę.

5.5 Pomiar metryk

System mierzy czas zgodnie z założeniami z wczesnych etapów (znaczniki t_1 – t_4):

- klient mierzy *czas obliczeń* $t_3 - t_2$ (`Stopwatch` wokół `Process`),
- serwer zna czas wysłania zadania t_1 (`SentAtUtc`) i czas odbioru wyniku t_4 ,
- *czas całkowity* $= t_4 - t_1$, *czas komunikacji* $= (t_4 - t_1) - (t_3 - t_2)$,
- *przepustowość* = liczba kandydatów / *czas obliczeń* [kandydatów/s].

Klient mierzy czas właściwych obliczeń stoperem wokół wywołania strategii ataku:

```
var stopwatch = Stopwatch.StartNew();
var found = _attackStrategy.Process(task.StartIndex, task.EndIndex, _targetHashes, ct);
stopwatch.Stop();

ct.ThrowIfCancellationRequested();

var result = new Result
{
    TaskId = task.TaskId,
    ComputeTimeMs = stopwatch.ElapsedMilliseconds
};
```

Listing 4: Pomiar czasu obliczeń po stronie klienta (`GrpcClient.ProcessAndSendResult`).

a serwer po odebraniu wyniku wyznacza czasy całkowity, komunikacji i przepustowość:

```
var totalTimeMs = Math.Max(1, (long)(t4Utc - task.SentAtUtc).TotalMilliseconds);
var computeTimeMs = Math.Max(1, result.ComputeTimeMs);
var communicationTimeMs = Math.Max(0, totalTimeMs - computeTimeMs);

var candidatesCount = task.EndIndex - task.StartIndex + 1;
var throughput = candidatesCount / (computeTimeMs / 1000.0);
```

Listing 5: Obliczanie metryk po stronie serwera (`PasswordBreakerService.SaveMetrics`).

Dla każdego ukończonego zadania serwer dopisuje wiersz do `metrics.csv` — m.in. `chunk_size`, `client_threads`, `connected_clients_at_result`, `compute_time_ms`, `communication_time_ms`, `throughput_candidates_per_sec`. Mierzenie czasu całkowitego po stronie serwera eliminuje problem rozjazdu zegarów między maszynami.

6 Metodologia eksperymentów i wyniki

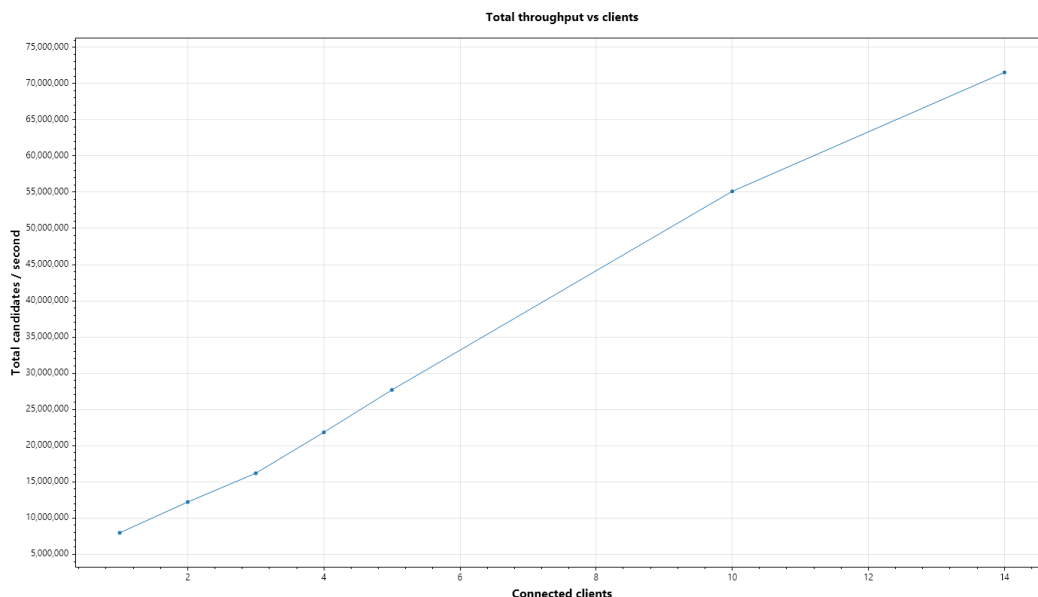
6.1 Metodologia

Przeprowadzono serię eksperymentów dla różnych liczb klientów oraz różnych wielkości chunków. Aby sprawiedliwie porównywać konfiguracje o różnej liczbie klientów, wprowadzono pojęcie **granulacji przypadającej na klienta**:

$$\text{granulacja na klienta} = \frac{\text{rozmiar chunku}}{\text{liczba klientów}}.$$

Każdy przebieg trwał ustalony czas, a liczbę klientów zwiększano do 14.

6.2 Wpływ liczby klientów

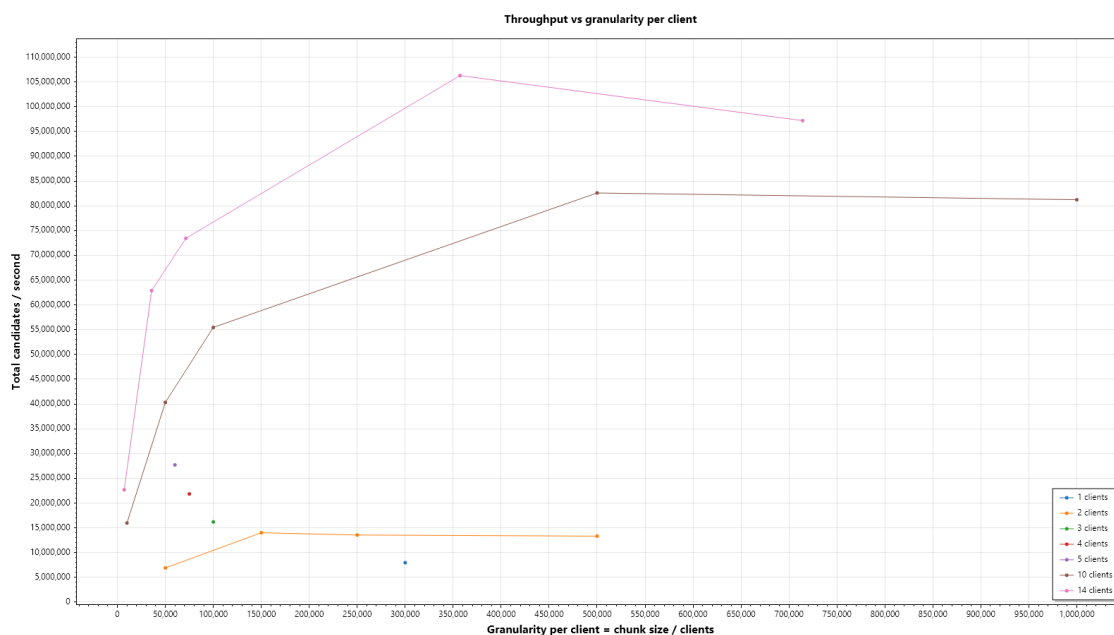


Rysunek 4: Całkowita przepustowość systemu w funkcji liczby podłączonych klientów.

Obserwacje: wraz ze wzrostem liczby klientów całkowita przepustowość rośnie niemal liniowo; najwyższą wydajność uzyskano dla 14 klientów.

Wniosek: dodawanie kolejnych węzłów zwiększa wydajność — system wykorzystuje równoległość obliczeń, a narzut koordynacji nie dominuje.

6.3 Wpływ granulacji i liczby klientów



Rysunek 5: Przepustowość w funkcji granulacji przypadającej na klienta, dla różnych liczb klientów.

Obserwacje: dla małych chunków dominuje narzut komunikacyjny (częste wymiany „zadanie/wynik”), więc przepustowość jest niższa. Zwiększanie granulacji początkowo wyraźnie podnosi wydajność, ale po przekroczeniu pewnego progu przyrost maleje (krzywa się wypłaszcza). Dla większej liczby klientów osiągnięta jest wyższa przepustowość.

Wniosek: przy zbyt małej granulacji narzut komunikacji jest zbyt duży i obniża przepustowość.

6.4 Wcześniej zauważone problemy i ich usunięcie

W trakcie prac (etap prototypu) zidentyfikowano i naprawiono ograniczenia wydajności:

- rezygnacja z zapisywania *wszystkich* generowanych skrótów (oszczędność pamięci),
- ograniczenie nadmiernego logowania w konsoli (mniejszy narzut),
- zwiększenie wykorzystania CPU dzięki wielowątkowemu przetwarzaniu zakresu po stronie klienta, tak aby dominował czas *właściwych* obliczeń, a nie operacji pomocniczych.

7 Wnioski końcowe

- Zbudowano działający system rozproszony w architekturze klient-serwer (gRPC, .NET), realizujący dynamiczny przydział pracy i zbieranie wyników.
- System **dobrze skaluje się** wraz ze wzrostem liczby klientów — przepustowość rośnie niemal liniowo.
- **Granulacja zadań** ma istotny wpływ na wydajność: zbyt małe zadania powodują nadmierny narzut komunikacyjny, przez co przepustowość spada.
- Najlepsze wyniki osiągnięto przy dużej liczbie klientów.
- Mechanizmy heartbeat, watchdog i timeout zadań zapewniają odporność na rozłączenia i zawieszenia klientów, a numeracja przebiegów i zadań chroni przed duplikatami wyników.
- Czas właściwych obliczeń stanowi dominującą część czasu całkowitego, co potwierdza poprawność optymalizacji wykonanych w trakcie projektu.